

ESCUELA COLOMBIANA DE INGENIERÍA
JULIO GARAVITO

LABORATORIO - SEMANA 5

Lab #4 - Collaborative Board: Architecture
Foundation

Backend modular, contratos y persistencia desacoplada

Asignatura	Arquitecturas de Software - ARSW
Periodo	2026-2
Producto acumulativo	ARSW Collaborative Architecture Board
Duración sugerida	Hasta 4 días después de la sesión práctica
Modalidad	Individual o parejas, según indicación docente
Entrada	Starter oficial del Lab #4
Salida	Base obligatoria para el Lab #5

Idea guía

Este laboratorio no busca demostrar que usted sabe construir una API REST. Busca demostrar que puede estructurar una aplicación pequeña con límites explícitos, dependencias controladas y evidencia arquitectónica que corresponda con el código.

1. Resultado esperado

Al terminar el laboratorio, el repositorio debe contener una primera versión funcional del backend del Collaborative Board. El código resultante será la línea base obligatoria del siguiente laboratorio.



Regla de continuidad

No cambie el dominio ni reorganice arbitrariamente los paquetes al finalizar. El Lab #5 partirá de esta misma estructura y añadirá el cliente web sobre los contratos definidos aquí.

2. Alcance funcional controlado

En esta fase un Board representa un plano simple y persistirá únicamente en memoria del proceso. No se requiere base de datos, autenticación, frontend funcional ni tiempo real.

- Crear un Board con nombre e identificador generado por el servidor.
- Consultar un Board existente por su identificador.
- Reemplazar el estado completo de un Board existente.
- Manejar inicialmente elementos de tipo RECTANGLE y TEXT.
- Responder de forma consistente cuando el Board no exista o el contrato de entrada sea inválido.

Fuera de alcance

WebSockets/STOMP, colaboración multiusuario, conectores gráficos, autenticación, base de datos externa, microservicios y despliegue. Estos elementos no deben agregarse en este laboratorio.

3. Starter entregado

El archivo ZIP contiene un proyecto Maven compilable con la estructura de paquetes, modelos base, puertos, adaptadores, controladores y pruebas preparadas con TODOs. El starter debe ser completado, no reemplazado por un proyecto generado desde cero.

```
src/main/java/.../collabboard
├── domain/model
│   ├── Board.java
│   ├── BoardElement.java
│   └── ElementType.java
├── application/port/out
│   └── BoardRepository.java
├── application/service
│   └── BoardApplicationService.java
├── infrastructure
│   ├── persistence/InMemoryBoardRepository.java
│   └── web/rest
│       ├── BoardRestController.java
│       ├── ApiError.java
│       └── GlobalExceptionHandler.java
```

4. Requisitos arquitectónicos obligatorios

ID	Requisito
RA-01	El controlador REST no almacena Boards ni implementa reglas de negocio.
RA-02	BoardApplicationService depende de la abstracción BoardRepository, nunca de InMemoryBoardRepository.
RA-03	InMemoryBoardRepository es un adaptador de infraestructura e implementa el puerto de salida.
RA-04	Las dependencias se suministran mediante inyección por constructor.
RA-05	Las excepciones internas se traducen a un contrato HTTP uniforme mediante un manejador centralizado.
RA-06	La estructura implementada debe coincidir con las vistas arquitectónicas entregadas.
RA-07	No se permite una clase “God Controller” ni acceso directo al Map desde componentes web.

5. Contrato REST mínimo

Método	Recurso	Propósito	Respuesta esperada
--------	---------	-----------	--------------------

Método	Recurso	Propósito	Respuesta esperada
POST	/api/boards	Crear Board	201 + Board creado
GET	/api/boards/{boardId}	Consultar Board	200 + Board
PUT	/api/boards/{boardId}	Reemplazar estado	200 + Board actualizado

El contrato exacto de request/response debe documentarse en ``docs/api-contract.md``. Se permite proponer pequeñas mejoras si no rompen el alcance del laboratorio y quedan justificadas.

6. Trabajo a realizar

6.1 Complete el dominio

Revise Board y BoardElement. Defina invariantes mínimas: identificadores no vacíos, nombre de Board válido y tipos de elemento soportados. No incluya lógica de HTTP en estas clases.

6.2 Complete el puerto de persistencia

Defina las operaciones mínimas que necesita la aplicación. Evite diseñar una interfaz como copia automática de un framework de base de datos.

6.3 Implemente el adaptador en memoria

Use una estructura simple en memoria. En esta semana NO se evaluará todavía el comportamiento concurrente de la colección; esa discusión regresará más adelante en el curso.

6.4 Implemente el Application Service

Centralice los casos de uso create, get y replace/update. El servicio debe depender del puerto BoardRepository.

6.5 Implemente el controlador REST

El controlador debe traducir HTTP hacia casos de uso. Manténgalo delgado.

6.6 Implemente manejo uniforme de errores

Use excepciones concretas y GlobalExceptionHandler. No retorne stack traces ni mensajes internos de Java al cliente.

6.7 Complete las pruebas

Incluya pruebas del servicio y del contrato REST. Una prueba debe demostrar el caso Board inexistente.

6.8 Documente la arquitectura

Actualice los artefactos descritos en la sección 8. Los diagramas deben representar la solución entregada, no una arquitectura ideal distinta del código.

7. Criterios de aceptación

- ☐ El proyecto ejecuta con Java 21 y Maven.
- ☐ ``mvn test`` finaliza correctamente.
- ☐ Es posible crear, consultar y reemplazar un Board mediante HTTP.
- ☐ Un Board inexistente produce un error HTTP coherente y un ApiError uniforme.
- ☐ El Application Service puede probarse sin levantar el servidor web.

- ☐ Cambiar el adaptador de persistencia no exige modificar el controlador REST.
- ☐ No hay referencias a clases de infraestructura dentro del paquete domain.
- ☐ Los diagramas reflejan nombres y dependencias observables en el código.

8. Evidencia arquitectónica obligatoria

8.1 Vista de Aplicación - ArchiMate

Entregue una vista sencilla, preferiblemente en Archi, Draw.io o la herramienta indicada por el docente. Debe mostrar como mínimo la interfaz REST, el servicio de aplicación, el repositorio/puerto y el almacenamiento en memoria, con relaciones comprensibles.

8.2 Diagrama de clases

Incluya únicamente las clases e interfaces que expliquen la estructura principal. Deben verse las dependencias entre BoardRestController, BoardApplicationService, BoardRepository e InMemoryBoardRepository, además del modelo relevante.

8.3 Mini ADR

Cree `docs/ADR-001-repository-boundary.md` con Contexto, Decisión, Consecuencias positivas, Trade-off y Evidencia.

8.4 Declaración de uso de IA

Complete `docs/AI\_USAGE.md`. Declarar el uso de IA no resta nota. Ocultarlo o entregar decisiones que el equipo no puede explicar sí afecta la evaluación.

Campo	Contenido esperado
Herramienta	Nombre de la herramienta utilizada
Actividad	Qué parte del trabajo apoyó
Prompt / propósito	Qué se solicitó y para qué
Validación	Cómo verificó el estudiante la respuesta
Cambios	Qué corrigió, rechazó o adaptó

9. Restricciones de diseño

- No introducir una base de datos externa.
- No implementar frontend funcional todavía.
- No agregar WebSockets/STOMP en esta fase.
- No usar Lombok como sustituto del razonamiento sobre el modelo.
- No sustituir la estructura por un framework o plantilla que esconda las dependencias evaluadas.
- No crear capas vacías únicamente para “cumplir” con una arquitectura en capas.

10. Entrega

Elemento	Ubicación / evidencia
Código fuente	Repositorio del estudiante, o el de sus grupos de trabajo.
Pruebas	`src/test/java`
Contrato REST	`docs/api-contract.md`

Elemento	Ubicación / evidencia
Vista ArchiMate	`docs/architecture/`
Diagrama de clases	`docs/architecture/`
ADR	`docs/ADR-001-repository-boundary.md`
Uso de IA	`docs/AI_USAGE.md`
README	Instrucciones claras para ejecutar y probar

Importante para la Semana 6

La entrega aprobada del Lab #4 será la entrada conceptual del Lab #5. El siguiente laboratorio añadirá un cliente web SVG que consumirá esta API. Cambiar contratos sin justificación generará trabajo adicional al propio equipo.

11. Rúbrica

Criterio	Peso
Estructura arquitectónica, responsabilidades y dirección de dependencias	35%
Aplicación correcta de DIP e inyección de dependencias	20%
Contrato REST y manejo consistente de errores	15%
Pruebas y evidencia de funcionamiento	15%
ArchiMate, diagrama de clases y ADR consistentes con el código	10%
README y declaración responsable de uso de IA	5%
TOTAL	100%

Pregunta de sustentación

Si mañana InMemoryBoardRepository se reemplaza por otro adaptador, ¿qué componentes deberían cambiar y cuáles deberían permanecer intactos? La respuesta debe coincidir con el código entregado.